An Improved Lower Bound for the $3x + 1$ Problem:
A Friendly Guide

Companion notes to "$\pi_a(x) \geq x^{0.895}$"*

June 12, 2026

1 The question

Nobody can prove that *every* number reaches 1 under the Collatz rule (halve if even, triple-and-add-one if odd). So mathematicians ask a humbler question with a provable answer:

Out of the first x numbers, how many can we PROVE reach 1?

Call that count $\pi_1(x)$. If the conjecture is true, $\pi_1(x) = x$. What can actually be proved has crept up over five decades, always in the form "at least x^β of them":

year	authors	exponent β
1978	Crandall	small positive
1989	Krasikov	0.43
1998	Wirsching	0.48
1995	Applegate–Lagarias	0.81
2003	Krasikov–Lagarias	0.84
2026	this work	**0.895**

To calibrate: $x^{0.895}$ of a trillion is about 52 billion, versus 13 billion for $x^{0.84}$ — and the dream, equivalent to "almost density one," would be $x^{0.999\cdots}$. The 2003 record stood for twenty-three years. The improvement here required, as the paper's acknowledgments put it, *no new mathematics — only twenty-three years of Moore's law and one change of algorithm.* These notes explain the method from scratch, because it is unusually pretty.

2 Growing the tree backwards

Instead of asking "does n reach 1?", turn the dynamics around: start at 1 and ask *which numbers arrive here?* Every number n has one guaranteed ancestor-in-reverse, namely $2n$ (because $2n$ halves to n). Some numbers have a second one: if $2n - 1$ is divisible by 3, then $b = (2n - 1)/3$ is an odd number whose triple-plus-one, halved, is exactly n.

*Written in collaboration with Claude (Anthropic). These notes explain the ideas; the paper contains the precise statements and the certification procedure. The Collatz conjecture itself remains open.

So the numbers that provably reach 1 form a **tree**: from each node, one branch always continues (doubling), and a second branch sprouts whenever a divisibility condition holds. Every node of the tree is a certified success — no conjecture needed. If the tree branches often enough, it contains exponentially many numbers, and counting them gives a lower bound for $\pi_1(x)$.

There is a tension, though. Doubling makes tree members *bigger*, so a tree with many members might spend them on numbers far beyond x. What matters is the count of members *below* x: roughly, (how fast the tree branches) measured against (how fast its members grow). That ratio is what becomes the exponent β.

Who gets the second child? The condition "$2n-1$ divisible by 3" depends only on the remainder of n when divided by 3. And the remainder of the child depends on the remainder of the parent when divided by 9, and so on: remainders modulo $3, 9, 27, \ldots, 3^k$ are the natural coordinates of the tree. Tracking finer remainders (larger k) gives a more detailed map of where branching is plentiful — and provably never a worse exponent. The entire fifty-year history in the table above is, in essence, the same tree examined at finer and finer resolution.

3 From tree to exponent: the certificate

Here is the 2003 framework, de-jargonized. Assign to every remainder class $m \pmod{3^k}$ a positive weight c_m — think of it as a prediction of how fertile that class's subtrees are. Krasikov and Lagarias wrote down a system of inequalities — one per class, each saying "this class's weight is justified by the weights of the classes its children land in, discounted by a growth rate λ" — and proved:

If weights satisfying the system exist for some rate λ, then $\pi_1(x) \geq x^{\log_2 \lambda}$.

The weights are called a **certificate**: a finite list of numbers that, once checked against the finitely many inequalities, proves an infinite statement. Bigger λ = better exponent; finer classes (bigger k) admit bigger λ. In 2003 the state of the art in solving such systems (linear programming) reached $k = 11$ — 59,049 classes — and $\lambda = 1.792$, i.e. the exponent 0.84.

4 The unlock: certificates that find themselves

The new observation is about the *shape* of the system. For a fixed rate λ, the inequalities say precisely

$$c \;\leq\; F(c),$$

where F takes the whole list of weights and produces a new list, each entry built from other entries by adding, scaling by positive constants, and taking minimums. Such an F has two golden properties: it is *monotone* (raise inputs, outputs never drop) and *scale-invariant* (double all inputs, outputs double).

For maps like that there is a classical trick — ironically associated with the same Lothar Collatz, in his day job as a numerical analyst. Start from any guess, apply F, rescale, repeat (readers who know how search engines rank pages have seen this dance: it is power iteration). The guesses settle toward an equilibrium, and the punchline is:

Any non-shrinking guess is a proof. You do not need to solve the system, trust the iteration, or invoke any theory: if at any moment your list satisfies $c \le F(c)$ entry-by-entry — "one more application of the rules doesn't shrink any weight" — then that very list is a valid certificate, checkable by anyone with a multiplication routine. The iteration is just a way of *finding* such a list; it plays no role in the proof.

One sweep of F costs time proportional to the number of classes. That replaces the 2002 bottleneck (general-purpose linear programming) and carries the computation from 59 thousand classes to 43 million ($k = 17$) on a laptop, with two further accelerations: each level's certificate is recycled as the starting guess for the next (the 2003 paper itself proves certificates lift between levels), and at the largest levels one simply aims at a target rate directly — a hit is a hit, no search required.

5 Why you can trust a 43-million-line proof

Floating-point arithmetic found the certificates; floating-point arithmetic is famously not to be trusted. So nothing in the final claim rests on it:

1. **Rational rates.** The claimed rate is chosen as $\lambda = 2^{p/q}$ for whole numbers p, q — so the exponent is *exactly* p/q (for the record: $179/200 = 0.895$). The system's coefficients involve irrational quantities; each is replaced by a nearby fraction that is provably *below* it, the proof being a single comparison of huge whole numbers (is $u^q \cdot 2^{2p} \le v^q \cdot 3^p$?). Using smaller coefficients only makes the inequalities harder to satisfy — errors can only hurt us, never help — so soundness leans one way.

2. **Exact verification.** The certificate is converted to whole numbers and every one of the 43,046,721 inequalities is checked in exact integer arithmetic. No rounding exists anywhere in the verified statement.

3. **Two independent referees.** The checker was implemented twice — the second time from a clean slate, re-deriving the inequality system from the published paper, using different number representations and different bounding tricks, sharing not a line of code. Both pass every level. And the pair was tested for blindness: corrupt a single entry of a certificate and both checkers catch exactly that violation.

4. **Validation against history.** Run at the 2003 levels, the machinery reproduces every entry of Krasikov–Lagarias' published table to all seven printed decimals, including their record 1.7922310.

The mathematical heavy lifting — the theorem that a certificate yields the bound at all — is exactly the published, peer-reviewed 2003 theorem, used verbatim. This work adds no asymptotic analysis; it adds certificates the 2003 authors could not compute, plus the means to check them beyond reasonable doubt.

6 The extra mile: a fragment proved by a machine, end to end

One can go one step further in trustworthiness: have a proof assistant verify not just the certificate but the *theorem* too. The companion repository contains a complete formalization, in Lean 4, of a simplified variant of the whole chain — the backward tree, the inequalities (proved from the dynamics, including the amusing fact that the tree can contain no loops because the orbit of 8 ends in the $1 \to 2 \to 1$ cycle), an 81-class certificate checked by Lean's own kernel, and the resulting bound

$$\pi_1(x) \;\geq\; \text{const} \cdot x^{0.4543},$$

with literally every logical step machine-checked. That was the first machine-verified density bound for this problem in history; it has since been overtaken in its own repository.

The simplification that made 0.4543 formalizable was a coarse grid: the tree was only examined at whole doublings, and each examination point cost growth. The second-generation formalization slices each doubling into **fifty** geometric steps, arranged so that the growth rate per step is the exact fraction $1261/1250$ — keeping every check a whole-number comparison a kernel can do — and pays for the finer grid with a bigger certificate: 2,187 classes, plus, for each class, an explicit small "witness" number whose journey to 1 the kernel replays move by move (at most 146 moves each). The result, again with every step machine-checked:

$$\pi_1(x) \;\geq\; \text{const} \cdot x^{0.63201}.$$

Why 0.63201 is a milestone. The number that rules this whole subject is $\log 2/\log 3 = 0.63093\ldots$ — the critical odd-step share that a number would have to sustain forever to escape to infinity, the constant in the cycle bounds, the line all the companion papers orbit around. The machine-verified exponent now sits *above* that constant. The two quantities measure different things (one counts successes, the other paces escapees), so this is a numerical milestone rather than a theorem about divergence — but there is something satisfying about the formally certified part of the subject overtaking its own critical constant.

Scaling the formalization up toward 0.895 is bookkeeping rather than new ideas.

7 What it would take to finish

Within this method, everything hinges on one open question: as the class resolution $k \to \infty$, does the best rate λ_k climb all the way to 2? If yes, the exponent climbs to 1 and *almost all* numbers provably reach 1 in the strongest counting sense. The data is consistent with it — the certified rates march $1.79, 1.81, 1.82, 1.83, 1.84, 1.85, 1.86$ with slowly shrinking steps — but each new level costs triple the memory and compute, and the limit question itself looks like genuine mathematics, not computation. Meanwhile the companion paper explains why no method that examines numbers through any bounded lens can settle the full conjecture: the last mile belongs to ideas not yet had.

8 Summary in four sentences

Count the numbers provably reaching 1 by growing the tree of ancestors of 1, organized by remainders modulo 3^k. A finite list of weights satisfying finitely many inequalities certifies an infinite bound $\pi_1(x) \geq x^{\log_2 \lambda}$ — and since the system is monotone and scale-invariant, certificates can be *found* by simple iteration and *verified* by exact arithmetic, no linear programming and no floating point in the final claim. Pushing from 59 thousand to 43 million classes lifts the twenty-three-year-old record from $x^{0.84}$ to $x^{0.895}$, checked by two independent verifiers and reproducing the historical table exactly. A simplified fragment is additionally machine-verified end to end in Lean — first at $x^{0.4543}$, now at $x^{0.63201}$, above the problem's critical constant $\log 2/\log 3 = 0.63093$.

Postscript (August 2026): the record, checked by machine

The 0.895 above rests on Krasikov and Lagarias's theorem plus an exact computer check. As of August 2026 a weaker but fully machine-verified version exists: the Lean proof assistant has checked, from the definition of the $3x+1$ map upward, that at least $x^{0.8569}$ of the integers below x reach 1 — above the twenty-year-old published record $x^{0.84}$ (a first run at a smaller level gave $x^{0.8476}$).

The obstacle was a single term in the Krasikov–Lagarias inequalities that looks "forward in time": one of the children in the tree of preimages is *smaller* than its parent, so its tree is measured at a larger scale. Induction wants to look backward. Krasikov and Lagarias got around this with an elimination procedure whose termination needs Kőnig's lemma. The formal proof gets around it differently: instead of inducting on scale alone, it inducts on the pair (scale, size of the root) combined into one number, $10t + \lfloor 498 \log_2 a \rfloor$, chosen so that every step of the recursion makes it strictly smaller — the forward-looking child is smaller by a factor $3/2$, and $498 \log_2(3/2) = 291.3$ just beats the 290 that the scale gains. That "just" is not luck: on the exact scale the two effects cancel precisely, and it is the deliberate $1/50$-step rounding of the scale that opens the gap.

The certificate — 531,441 integers, one per congruence class mod 3^{13} — is stored inside the proof as 243 very large numbers and checked, class by class, by Lean's kernel in a little over two hours. Nothing is trusted from outside.

Credit where due: a separate project (`Menta2357/collatz-classical`, July 2026) had already formalized Krasikov and Lagarias's own argument in Lean and reproduced their $x^{0.84}$ exactly. The result here is the first to go *past* that number with a machine-checked proof, and it does so with a different, shorter argument.